

ValueError: Exceeds the limit (4300) for integer string conversion

Asked 2 years, 4 months ago Modified 3 months ago Viewed 40k times

▲

▼

🔖

🔄

28

```
>>> import sys
>>> sys.set_int_max_str_digits(4300) # Illustrative, this is the default.
>>> _ = int('2' * 5432)
Traceback (most recent call last):
...
ValueError: Exceeds the limit (4300) for integer string conversion: value has
5432 digits.
```

Python 3.10.7 introduced this breaking change for type conversion.

Documentation: [Integer string conversion length limitation](#)

Actually I don't understand why

1. this was introduced and
2. where does the default value of 4300 come from? Sounds like an arbitrary number.

`python` `python-3.x` `python-3.10` [Edit tags](#)

[Share](#) [Edit](#) [Follow](#) [Close](#) [Flag](#)

edited Apr 17, 2023 at 6:46
 Talha Tayyab
27k ● 48 ● 115 ● 151

asked Sep 12, 2022 at 17:30
 Data Mastery
2,075 ● 4 ● 30 ● 80

- 3 ▲
- The page you linked to explains the rationale. I suspect the default value is related to the size of some internal array being a multiple of 1024. – [Barmar](#) Sep 12, 2022 at 17:36
- 3 ▲
- I found this in the CPython source: "Default int base conversion size limitation: [...] Chosen such that this isn't wildly slow on modern hardware and so that everyone's existing deployed numpy test suite passes [...] 4300 decimal digits fits a ~14284 bit number." – [Lone Lunatic](#) Sep 12, 2022 at 17:42

5 Answers

Sorted by: [Reset to default](#) [Date modified \(newest first\)](#) ⌵

▲

▼

🔖

🔄

2

The [Python documentation](#) contains this brief explanation:

Limiting conversion size offers a practical way to avoid [CVE-2020-10735](#).

... and later on

The default `sys.int_info.default_max_str_digits` is expected to be reasonable for most applications.

The other answers here and [this comment](#) already contain more details if you are curious, but I included these passages to make this answer self-contained.

If you need to raise the limit, please understand that Python needs to compile your code before it can be run. If you embed a number which exceeds this limit, you get a syntax error, even if you attempt to raise the limit within your script.

```
# XXX BUGGY, see text
import sys
sys.set_int_max_str_digits(0)
bigone =
141592653589793238462643383279502884197169399375105820974944592307816406286208998628
```

Traceback:

```
File "/tmp/bigun.py", line 3

27211166039666587309254711055785376346682065310989652691862056476931257058635662018:
SyntaxError: Exceeds the limit (4300 digits) for integer string conversion: value
has 4302 digits; use sys.set_int_max_str_digits() to increase the limit -
Consider hexadecimal for huge integer literals to avoid decimal conversion
limits.
```

As demonstrated in several examples in the documentation, the workaround for this is to embed the number as a string, then convert it at run time (so that `sys.set_int_max_str_digits()` actually gets run before Python attempts to parse the big number):

```
import sys
sys.set_int_max_str_digits(0)
bigone =
int("1415926535897932384626433832795028841971693993751058209749445923078164062862089
```

You can also override this setting via an environment variable; at the shell prompt, this would look like

```
PYTHONINTMAXSTRDIGITS=0 python yourscript.py
```

or setting the `-X` option:

```
python -X int_max_str_digits= yourscript.py
```

[Share](#) [Edit](#) [Follow](#) [Flag](#)

edited Oct 8, 2024 at 6:01
 John James 1st Interveno Milita
5 ● 2

answered Oct 7, 2023 at 8:34
 tripleee
189k ● 36 ● 312 ● 360

▲ Adding to the existing answers, if you need to dynamically change the value, depending on what kind of input you handle, you may want to check its current value with `sys.get_int_max_str_digits()`:

Vote



```
>>> sys.get_int_max_str_digits()
4300
>>> sys.set_int_max_str_digits(8600)
>>> sys.get_int_max_str_digits()
8600
# do something with the new-gained freedom
# ...
# and if you want to set it back to default
>>> sys.set_int_max_str_digits(sys.int_info.default_max_str_digits)
>>> sys.get_int_max_str_digits()
>>> 4300
```

Share Edit Follow Flag

answered Apr 3, 2024 at 0:20

 **sm**
1,261 ● 2 ● 10 ● 18



if you get this error:

25

ValueError: Exceeds the limit (4300) for integer string conversion



you can increase the limit by:



```
import sys
sys.set_int_max_str_digits(0)
```

Now, you can do bigger calculations.

Link to documentation:

<https://docs.python.org/3/library/stdtypes.html#integer-string-conversion-length-limitation>

Share Edit Follow Flag

answered Jan 18, 2023 at 16:44

 **Talha Tayyab**
27k ● 48 ● 115 ● 151



See github issue [CVE-2020-10735: Prevent DoS by large int<->str conversions #95778](#):

12

Problem

A Denial Of Service (DoS) issue was identified in CPython because we use binary bignum's for our int implementation. A huge integer will always consume a near-quadratic amount of CPU time in conversion to or from a base 10 (decimal) string with a large number of digits. No efficient algorithm exists to do otherwise.

It is quite common for Python code implementing network protocols and data serialization to do `int(untrusted_string_or_bytes_value)` on input to get a numeric value, without having limited the input length or to do `log("processing thing id %s", unknowingly_huge_integer)` or any similar concept to convert an int to a string without first checking its magnitude. (`http`, `json`, `xmlrpc`, `logging`, loading large values into integer via linear-time conversions such as hexadecimal stored in yaml, or anything computing larger values based on user controlled inputs... which then wind up attempting to output as decimal later on). All of these can suffer a CPU consuming DoS in the face of untrusted data.

Everyone auditing all existing code for this, adding length guards, and maintaining that practice everywhere is not feasible nor is it what we deem the vast majority of our users want to do.

This issue has been reported to the Python Security Response Team multiple times by a few different people since early 2020, most recently a few weeks ago while I was in the middle of polishing up the PR so it'd be ready before 3.11.0rc2.

Mitigation

After discussion on the Python Security Response Team mailing list the conclusion was that we needed to limit the size of integer to string conversions for non-linear time conversions (anything not a power-of-2 base) by default. And offer the ability to configure or disable this limit.

The Python Steering Council is aware of this change and accepts it as necessary.

Further discussion can be found on the Python Core Developers Discuss thread [int/str conversions broken in latest Python bugfix releases](#).

I found [this comment](#) by Steve Dower to be informative:

Our apologies for the lack of transparency in the process here. The issue was first reported to a number of other security teams, and converged in the Python Security Response Team where we agreed that the correct fix was to modify the runtime.

The delay between report and fix is entirely our fault. The security team is made up of volunteers, our availability isn't always reliable, and there's nobody "in charge" to coordinate work. We've been discussing how to improve our processes. However, we did agree that the potential for exploitation is high enough that we didn't want to disclose the issue without a fix available and ready for use.

We did work through a number of alternative approaches, implementing many of them. The code doing `int(gigabyte_long_untrusted_string)` could be anywhere inside a `json.load` or HTTP header parser, and can run very deep. Parsing libraries are everywhere, and tend to use `int` indiscriminately (though they usually handle `ValueError` already). Expecting every library to add a new argument to every `int()` call would have led to thousands of vulnerabilities being filed, and made it impossible for users to ever trust that their systems could not be DoS'd.

We agree it's a heavy hammer to do it in the core, but it's also the only hammer that has a chance of giving users the confidence to keep running Python at the boundary of their apps.

Now, I'm personally inclined to agree that `int->str` conversions should do something other than raise. I was outvoted because it would break round-tripping, which is a reasonable argument that I accepted. We can still improve this over time and make it more usable. However, in most cases we saw, rendering an excessively long string isn't desirable either. That should be the opt-in behaviour.

Raising an exception from `str` may prove to be too much, and could be reconsidered, but we don't see a feasible way to push out updates to every user of `int`, so that will surely remain global.

Share Edit Follow Flag

edited Sep 12, 2022 at 17:44

answered Sep 12, 2022 at 17:36

 **Steven Rumbalski**
45.5k ● 10 ● 94 ● 124



0



This post is hidden. It was [deleted](#) 2 years ago by the post author.

This is a mitigation of a vulnerability [CVE-2020-10735](#)

<https://github.com/python/cpython/issues/95778>

I don't know why exactly 4300 digits, but any limit would be arbitrary. Given the number of atoms in the universe, it is a sufficiently huge number IMO.

Share Edit Follow **Undelete** Flag

answered Sep 12, 2022 at 17:37

VPFB
17.2k ● 7 ● 50 ● 90

While any number would be arbitrary, it seems strange that it's not a multiple of 1000 or 1024 -- the weird number begs for an explanation. – [Barmar](#) Sep 12, 2022 at 17:39

Comments disabled on deleted / locked posts / reviews